
Cours Modélisation et Vérification des Systèmes Informatiques (MVSI)

Modélisation, spécification et vérification CM3

Dominique Méry
Telecom Nancy, Université de Lorraine

Année universitaire 2024-2025

- ① Summation of the n first integers
- ② Principe(s) d'induction
- ③ Méthode de preuves de propriétés d'invariance

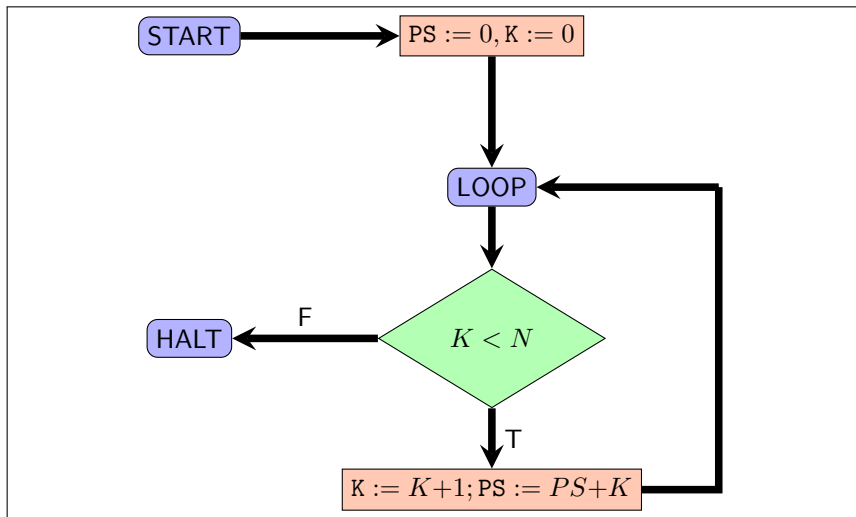
- ▶ Un programme ou un algorithme peuvent être annotés ou commentés.
- ▶ Un commentaire est une information pertinente destinée à être vue ou lue et qui a une importance relative dans l'esprit du concepteur.
- ▶ Un commentaire indique une information sur les données, sur les variables et donc sur l'état supposé du programme à l'exécution.
- ▶ Un commentaire est une annotation du texte du code qui nous permet de communiquer une information sémantique :
 - *à ce point, la variable k est plus petite sur n*
 - *l'indice e fait référence à une adresse licite de t et cette valeur est toujours positive*
 - *la somme des variables est positive*
- ▶ Les annotations peuvent être systématisées et obéir à une syntaxe spécifique définissant le langage d'annotations ;

```
/*@ assert l1:  z >= 3 && y == 3; */  
z = z + y;  
/*@ assert l2:  z >= 6 && y == 3; */
```

Calculer la somme des n premiers entiers (v0)

```
int fS(int n) {  
    int ps = 0;  
    int k = 0;  
    while (k < n) {  
        k = k + 1;  
        ps = ps + k;  
    };  
    return ps;  
}
```

Calculer la somme des n premiers entiers (flowchart)



Calculer la somme des n premiers entiers (v0)

```
// pre n >= 0;  
// post ps == n*(n+1) / 2;
```

```
int fS(int n) {  
    int ps = 0;  
    int k = 0;  
    while (k < n) {  
        k = k + 1;  
        ps = ps + k;  
    };  
    return ps;  
}
```

```
int main()  
{  
  
}
```

Calculer la somme des n premiers entiers (v0)

```
// pre  $n \geq 0$ ;  
// post  $ps == n * (n + 1) / 2$ ;
```

```
int fS(int n) {  
    int ps = 0;  
    int k = 0;  
    while (k < n) {  
        //  $ps = k * (k + 1) / 2$ ;  
        k = k + 1;  
        ps = ps + k;  
    };  
    //  $ps = n * (n + 1) / 2$ ;  
    return ps;  
}
```

```
int main()  
{
```

Calculer la somme des n premiers entiers (v0)

```
#include <stdio.h>

int fS(int n) {
    int ps = 0;
    int k = 0;
    while (k < n) {
        k = k + 1;
        ps = ps + k;
    };
    return ps;
}

int main()
{
    int z = 3;
    printf("Value for z=%d is %d\n", z, fS(z));
    return 0;
}
```

Definition of S(n) and IS(n)

$$\forall n \in \mathbb{N} : S(n) = \sum_{k=0}^n k$$

$$\begin{cases} IS(0) = 0 \\ n \geq 0, IS(n+1) = IS(n) + (n+1) \end{cases}$$

Property for S(n) and IS(n)

$$\forall n \in \mathbb{N} : S(n) = IS(n)$$

- ▶ base 0 : $S(0) = 0 = IS(0)$
- ▶ induction $i+1 : \forall j \in 0..i : S(j) = IS(j)$
 - $S(i+1) = \sum_{k=0}^{i+1} k$ (definition)
 - $S(i+1) = (\sum_{k=0}^i k) + i+1$ (property of summation)
 - $S(i+1) = S(i) + (i+1)$ (by definition of S(i))
 - $S(i+1) = IS(i) + (i+1)$ (by inductive assumption on S(i) et IS(j))
 - $S(i+1) = IS(i) + (i+1) = IS(i+1)$ (by definition of IS(i+1))
- ▶ conclusion : $\forall i \in \mathbb{N} : S(i) = IS(i)$ (by induction principle)

- ▶ $\forall n \in \mathbb{N} : S(n) = \sum_{k=0}^n k$
- ▶ $\left[\begin{array}{l} IS(0) = 0 \\ n \geq 0, IS(n+1) = IS(n) + (n+1) \end{array} \right.$
- ▶ $\forall n \in \mathbb{N} : S(n) = IS(n)$
- ▶
 - base 0 : $S(0) = 0$
 - induction $k+1$: $S(k+1) = S(k) + k + 1$
 - step $k+1$: $S(k+1) = S(k) + k + 1$
- ▶ $S(k) = ps$: current value of ps is $S(k)$
- ▶ $S(k-1) = ops$: current value of ops is $S(k-1)$
- ▶ step $k+1$: $ps = D(k+1) = S(k) + k + 1 = ops + k + 1$

```
#include <stdio.h>

int fS(int n) {
    int ps = 0;
    int k = 0;
    int ok=k, ops = 0;
    while (k < n) {
        ok=k;ops=ps;
        k = ok + 1;
        ps = ops + k;
    };
    return ps;
}

int main()
{
    int    z = 3;
    printf(" Value - for - z=%d - is -%d\n" , z , fS(z));
    return 0;
}
```

```
int fS(int n) {  
    int ps = 0;  
    int k = 0;  
    int ok=k, ops = 0;  
    while (k < n) {  
        /*@ assert 0 <= k && k < n  
           && ps == S(k) && ops == S(ok);    */  
        ok=k; ops=ps;  
        k = ok + 1;  
        ps = ops + k;  
        /*@ assert 0 <= k && k <= n && ps == S(k)  
           && ops == S(ok);    */  
    };  
    return ps;  
}
```

Calculer la somme des n premiers entiers

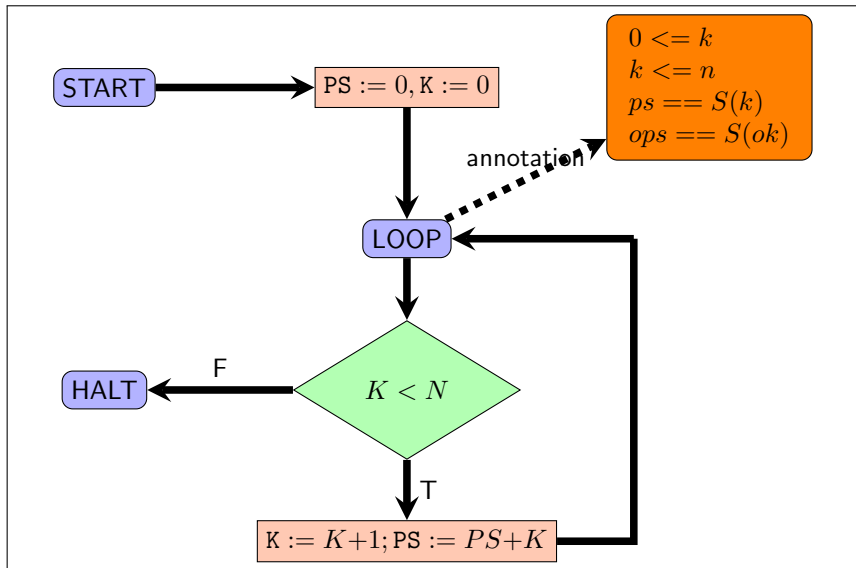
```
/*@ axiomatic S {
  @ logic integer S(integer n);
  @ axiom S_0: S(0) == 0;
  @ axiom S_i: \forall integer i; i > 0 ==> S(i) == S(i-1)+i;
  @ } */

/*@ requires n >= 0;
    assigns \nothing ;
    ensures \result == S(n);
*/
int fS(int n) {
  int ps = 0;
  int k = 0;
  int ok=k, ops=ps;
  /*@ loop invariant 0 <= k && k <= n && ps == S(k) && ops == S(ok) ;
      loop assigns ps, k, ops, ok;
  */
  while (k < n) {
    /*@ assert I0: 0 <= k && k < n && ps == S(k) && ops == S(ok); */
    ops=ps; ok=k;
    k = ok + 1;
    ps = ops + k;
    /*@ assert I1: 0 <= k && k <= n && ps == S(k) && ops == S(ok); */
  };
  /*@ assert ps == S(n); */
  return ps;}

```

```
frama-c -wp sum.c
[kernel] Parsing sum.c (with preprocessing)
[wp] Warning: Missing RTE guards
[wp] 8 goals scheduled
[wp] Proved goals:      8 / 8
Qed:                  6 (2ms-4ms-10ms)
Alt-Ergo 2.3.3:       2 (8ms-15ms) (38)
```

Calculer la somme des n premiers entiers (annotated flowchart)



- ▶ Définition des fonctions mathématiques nécessaires pour exprimer le calcul de la somme des n premiers nombres entiers.
- ▶ Expression des résultats intermédiaires appelés *sommes partielles*
- ▶ Relation entre la preuve par induction et la forme du corps de l'itération.
- ▶ Induction et calcul sont liés.

- ▶ Définition des fonctions mathématiques nécessaires pour exprimer le calcul de la somme des n premiers nombres entiers.
- ▶ Expression des résultats intermédiaires appelés *sommes partielles*
- ▶ Relation entre la preuve par induction et la forme du corps de l'itération.
- ▶ Induction et calcul sont liés.

$$x_0 \xrightarrow{P} x \quad (1)$$

$$x_0 \xrightarrow{\star} x \quad (2)$$

$$x_0 \longrightarrow x_1 \longrightarrow \dots \longrightarrow x \quad (3)$$

$$x_0 \longrightarrow x_1 \longrightarrow \dots \longrightarrow x_n \xrightarrow{step} x \quad (4)$$

- 1 Summation of the n first integers
- 2 Principe(s) d'induction
- 3 Méthode de preuves de propriétés d'invariance



On convient des notations suivantes équivalentes :
 $x \in E$ est équivalent à $E(x)$ pour toute valeur $x \in \mathbf{Vals}$.
Cette simplification permet relier un ensemble $U \subseteq \mathbf{Vals}$ à une assertion $U(x)$ en considérant que $U(x)$ et $x \in U$ désigne le même concept.

Les deux expressions suivantes sont équivalentes :

- ▶ $\forall x_0, x \in \mathbf{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow A(x)$
- ▶ $\forall x \in \mathbf{VALS}. (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{Next}^*(x_0, x)) \Rightarrow A(x)$

- ▶ $\forall x_0, x \in \mathbf{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow A(x)$
- ▶ $\forall x \in \mathbf{VALS}. (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{Next}^*(x_0, x)) \Rightarrow A(x).$
- ▶ $\text{REACHABLE}(M) = \{u | u \in \mathbf{VALS} \wedge (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{Next}^*(x_0, u))\}$ est l'ensemble des états accessibles à partir des états initiaux et on doit montrer la propriété de sûreté $A(x)$ en montrant l'inclusion des ensembles (model-checking) :
$$\text{REACHABLE}(M) \subseteq \{u | u \in \mathbf{VALS} \wedge A(u)\}$$

Soit $(Th(s, c), x, VALS, Init(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système S.

Une propriété $A(x)$ est une propriété de sûreté pour le système S,

$$\forall x_0, x \in VALS. Init(x_0) \wedge Next^*(x_0, x) \Rightarrow A(x)$$

si et seulement si,

il existe une propriété d'état $I(x)$, telle que :

$$\forall x, x' \in VALS : \begin{cases} (1) \text{ } Init(x) \Rightarrow I(x) \\ (2) \text{ } I(x) \Rightarrow A(x) \\ (3) \text{ } I(x) \wedge Next(x, x') \Rightarrow I(x') \end{cases}$$

La propriété $I(x)$ est appelée un invariant inductif de S et est une propriété de sûreté particulière plus forte que les autres propriétés de sûreté.

Soit une propriété $I(x)$ telle que :

$$\forall x, x' \in \text{VALS} : \begin{cases} (1) \text{ Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{Next}(x, x') \Rightarrow I(x') \end{cases}$$

Alors $A(x)$ est une propriété de sûreté pour le système S modélisé par M .

Soient x_0 et $x \in \text{VALS}$ tels que $\text{Init}(x_0) \wedge \text{Next}^*(x_0, x)$.

- ▶ On peut construire une suite telle que :

$$x_0 \xrightarrow{\text{Next}} x_1 \xrightarrow{\text{Next}} x_2 \xrightarrow{\text{Next}} \dots \xrightarrow{\text{Next}} (x_i = x).$$

- ▶ L'hypothèse (1) nous permet de déduire $I(x_0)$.
- ▶ L'hypothèse (3) nous permet de déduire $I(x_1), I(x_2), I(x_3), \dots, I(x_i)$. En utilisant l'hypothèse (2) pour x , nous en déduisons que x satisfait A .

$$\forall x_0, x \cdot x, y \in \text{VALS} \wedge \text{Init}(x_0) \wedge x_0 \xrightarrow[\text{Next}]{*} x \Rightarrow A(x)$$

PROUVONS QUE : il existe une propriété $I(x)$ telle que :

$$\forall x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{Next}(x, x') \Rightarrow I(x') \end{cases}$$

- ▶ Nous considérons la propriété suivante :

$$I(x) \hat{=} \exists x_0 \in \text{VALS} \cdot \text{Init}(x_0) \wedge x_0 \xrightarrow[\text{Next}]{*} x.$$

- ▶ $I(x)$ exprime que la valeur x est accessible à partir d'une valeur initiale x_0 .
- ▶ Les trois propriétés sont simples à vérifier pour $I(x)$. $I(x)$ est appelé le plus fort invariant du système de transition défini par Init et Next

- ▶ P. et R. Cousot développent une étude complète des propriétés d'invariance et de sûreté en mettant en évidence correspondances entre les différentes méthodes ou systèmes proposées par Turing, Floyd, Hoare, Wegbreit, Manna ... et reformulent les principes d'induction utilisés pour définir ces méthodes de preuve (voir les deux cubes des 16 principes).
- ▶ Deux types de principes sont proposés : assertionnel et relationnel.
- ▶ Nous utilisons l'expression de propriété de sûreté, alors que généralement il s'agit d'une propriété d'invariance ($\square$ propriété) et d'invariant au lieu d'invariant inductif.

.....

☒ Definition(assertion)

Soit $(Th(s, c), X, VALS, Init(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété assertionnelle de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge Next^*(x_0, x) \Rightarrow A(x).$$

.....

.....

☒ Definition(relation)

Soit $(Th(s, c), X, VALS, Init(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété R est une propriété relationnelle de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge Next^*(x_0, x) \Rightarrow R(x_0, x).$$

.....

- ▶ $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow R(x_0, x)$ (R) est une propriété relationnelle de sûreté.
- ▶ Soit $y = (x_0, x)$, $y_0 = (x_0, x_0)$, et
 $\text{NextR}(y, y') \stackrel{\text{def}}{=} \text{Next}(x, x') \wedge y = (x_0, x) \wedge y' = (x_0, x')$
- ▶ (R) est réécrit comme suit :
$$\forall y_0, y \in \text{VALS} \times \text{VALS}. \text{Init}(x_0) \wedge y_0 = (x_0, x_0) \wedge \text{NextR}^*(y_0, y) \Rightarrow R(y) \text{ (R)}$$
- ▶ Par la propriété de correction et de complétude
- ▶ il existe une propriété d'état $IR(y)$, telle que :

$$\forall y_0, y \in \text{VALS} \times \text{VALS}. \begin{cases} (1) \text{Init}(x_0) \wedge y_0 = (x_0, x_0) \Rightarrow IR(y) \\ (2) IR(y) \Rightarrow R(y) \\ (3) IR(y) \wedge \text{NextR}(y, y') \Rightarrow IR(y') \end{cases}$$

- ▶ il existe une propriété relationnelle $IR(x_0, x)$, telle que :

$$\forall x_0, x \in \text{VALS}. \begin{cases} (1) \text{Init}(x_0) \wedge y_0 = (x_0, x_0) \Rightarrow IR(x_0, x_0) \\ (2) IR(x_0, x) \Rightarrow R(x_0, x) \\ (3) IR(x_0, x) \wedge \text{Next}(x, x') \Rightarrow IR(x_0, x') \end{cases}$$

On obtient donc deux types de principes d'induction selon les propriétés de sûreté

Complétude et correction

$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow A(x).$

si, et seulement si,

il existe $I \in \mathcal{P}(\text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow I(x_0) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{Next}(x, x') \Rightarrow I(x') \end{cases}$$

- ▶ L'absence d'erreurs à l'exécution est caractérisée comme une propriété assertionnelle, puisqu'elle porte sur le fait qu'un état est sans erreurs à l'exécution si les calculs sont définis en cet état.
- ▶ $A\text{-RTE}(x)$ signifie qu'on peut évaluer la relation $\text{Next}(x, x')$ dans le modèle de calcul : $A\text{-RTE}(x) = \exists x' \in \mathcal{P}(\text{VALS}). \text{Next}(x, x')$

Complétude et correction

$$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow A(x_0, x).$$

si, et seulement si,

il existe $R \in \mathcal{P}(\text{VALS} \times \text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow R(x_0, x_0) \\ (2) R(x_0, x) \Rightarrow A(x_0, x) \\ (3) R(x_0, x) \wedge \text{Next}(x, x') \Rightarrow R(x_0, x') \end{cases}$$

- ▶ La correction partielle est caractérisée comme une relation entre l'état initial et l'état courant.
- ▶ $A-PC(x)$ signifie que le programme se termine dans des états définissant la postcondition dans le modèle de calcul :
 $A-PC(x_0, x) = \text{term}(x) \Rightarrow \text{PRE}(x_0) \wedge \text{POSTCOND}(x_0, x).$

Principe assertionnel de sûreté ou d'invariance

$$\exists I(x) \in \mathcal{P}(\text{VALS}). \left[\begin{array}{l} \forall x, x' \in \text{VALS}. \\ (1) \text{ Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{Next}(x, x') \Rightarrow I(x') \end{array} \right.$$

Principe relationnel de sûreté ou d'invariance

$$\exists IR(x_0, x) \in \mathcal{P}(\text{VALS} \times \text{VALS}). \left[\begin{array}{l} \forall x_0, x, x' \in \text{VALS}. \\ (1) \text{ Init}(x_0) \Rightarrow IR(x_0, x_0) \\ (2) IR(x_0, x) \Rightarrow R(x_0, x) \\ (3) IR(x_0, x) \wedge \text{Next}(x, x') \Rightarrow IR(x_0, x') \end{array} \right.$$

- 1 Summation of the n first integers
- 2 Principe(s) d'induction
- 3 Méthode de preuves de propriétés d'invariance

Un programme P *remplit* un contrat $(pre, post)$:

- ▶ P transforme une variable x à partir d'une valeur initiale x_0 et produisant une valeur finale x_f : $x_0 \xrightarrow{P} x_f$
- ▶ x_0 satisfait pre : $pre(x_0)$ and x_f satisfait $post$: $post(x_0, x_f)$
- ▶ $pre(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow post(x_0, x_f)$

requires $pre(x_0)$

ensures $post(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

▶ $pre(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$

▶ $pre(x_0) \wedge P_f(x_0, x) \Rightarrow post(x_0, x)$

▶ conditions de vérification pour toutes les paires $\ell \longrightarrow \ell'$

- ▶ on définit un ensemble de points de contrôle LOCATIONS
- ▶ pour chaque programme ou algorithme P , LOCATIONS est un ensemble fini de valeurs et une variable cachée notée pc parcourt cet ensemble selon l'enchaînement.
- ▶ l'espace des valeurs possibles VALS est un produit cartésien de la forme $\text{LOCATIONS} \times \text{MEMORY}$
- ▶ les variables x du système se décomposent en deux entités indépendantes $x = (pc, v)$ avec comme conditions $pc \in \text{LOCATIONS}$ et $v \in \text{MEMORY}$.

$$x = (pc, v) \wedge pc \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \quad (5)$$

On considère un programme P annoté ; on se donne un modèle relationnel

$\mathcal{MP} = (Th(s, c), x, \text{VALS}, \text{Init}(x), \{r_0, \dots, r_n\})$ où

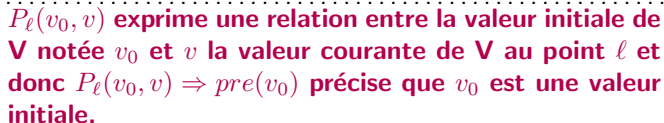
- ▶ $Th(s, c)$ est une théorie définissant les ensembles, les constantes et les propriétés statiques de ce programme
- ▶ x est une liste de variables flexibles et x comprend une partie contrôle et une partie mémoire.
- ▶ $\text{LOCATIONS} \times \text{MEMORY}$ est un ensemble de valeurs possibles pour x .
- ▶ $\{r_0, \dots, r_n\}$ est un ensemble fini de relations reliant les valeurs avant x et les valeurs après x' et conformes à la relation de succession $\longrightarrow$ entre les points de contrôle.
- ▶ $\text{Init}(x)$ définit l'ensemble des valeurs initiales de (pc_0, v) et $x = (pc_0, v)$ avec $pre(v)$ qui caractérise les valeurs initiales de v au point initial.

.....

$$\ell_1 \longrightarrow \ell_2 \stackrel{def}{=} pc = \ell_1 \wedge pc' = \ell_2$$

Soit une structure $(\text{LOCATIONS}, \longrightarrow)$ et une étiquette $\ell \in \text{LOCATIONS}$.

.....



- ▶ Les étiquettes ℓ appartiennent à `LOCATIONS` : $\ell \in \text{LOCATIONS}$.
- ▶ Les variables v appartiennent à `MEMORY` : $v \in \text{MEMORY}$.
- ▶ $pre(v_0)$ spécifie les valeurs initiales de v .
- ▶ Chaque fois que le contrôle est en ℓ , v satisfait $P_\ell(v)$:
 $pc = \ell \Rightarrow P_\ell(v_0, v)$.
- ▶ A tout état (ℓ, v) du programme, la propriété suivante est vraie mais doit être prouvée :

$$J(\ell_0, v_0, pc, v) \stackrel{def}{=} \left[\begin{array}{l} \wedge pc \in \text{LOCATIONS} \\ \wedge v \in \text{MEMORY} \\ \dots \\ \wedge pc = \ell \Rightarrow P_\ell(v_0, v) \\ \dots \end{array} \right]$$

- ▶ $J(\ell_0, v_0, pc, v)$ est un invariant construit à partir des annotations produites mais il faut montrer que cet invariant permet de vérifier les trois conditions du principe d'induction.



ℓ_0 désigne l'étiquette marquant le début de l'algorithme et ℓ_f est la fin du programme. On pourra utiliser simplement 0 et f.

$$x = (pc, v) \text{ et } J(\ell_0, v_0, pc, v) \stackrel{def}{=} \left[\begin{array}{l} \wedge pc \in \text{LOCATIONS} \\ \wedge v \in \text{MEMORY} \\ \dots \\ \wedge pc = \ell \Rightarrow P_\ell(v_0, v) \\ \dots \end{array} \right.$$

Soit $(Th(s, c), x, \text{VALS}, \text{Init}(x), \{r_0, \dots, r_n\})$ un modèle relationnel pour ce programme. Une propriété $A(x_0, x)$ est une propriété de sûreté pour P , si $\forall x_0, x \in \text{LOCATIONS} \times \text{MEMORY}. \text{Init}(x_0) \wedge x_0 \xrightarrow{\text{Next}} x \Rightarrow A(x)$. On sait que cette propriété implique qu'il existe une propriété d'état $I(x_0, x)$ telle que les trois propriétés sont vérifiées mais on applique cette vérification pour J :

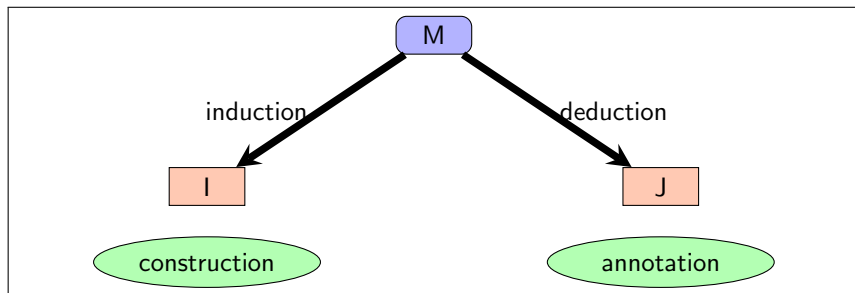
$\forall x_0, x, x' \in \text{LOCATIONS} \times \text{MEMORY} :$

- $$\left\{ \begin{array}{l} (1) \text{ Init}(x_0) \Rightarrow J(x_0, x_0) \\ (2) J(x_0, x) \Rightarrow A(x_0, x) \\ (3) \forall i \in \{0, \dots, n\} : J(x_0, x) \wedge x \ r_i \ x' \Rightarrow J(x_0, x') \end{array} \right.$$

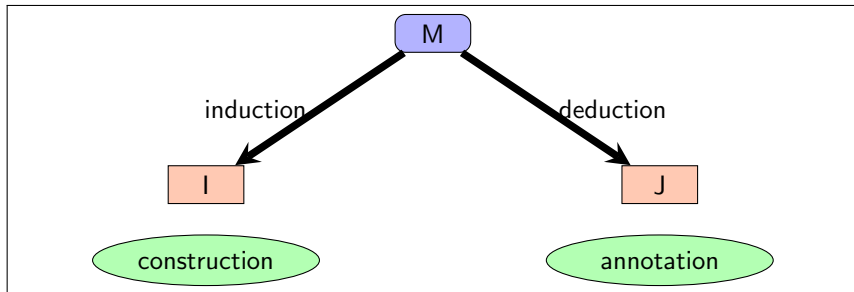


$\forall i \in \{0, \dots, n\} : J(x_0, x) \wedge x \ r_i \ x' \Rightarrow J(x_0, x')$ est équivalent à $J(x_0, x) \wedge (\exists i \in \{0, \dots, n\} : x \ r_i \ x') \Rightarrow J(x_0, x')$

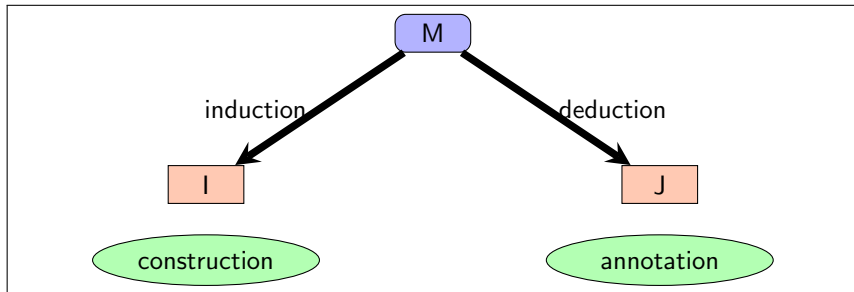
- ▶ Application de la correction du principe relationnel d'induction : si on vérifie les trois propriétés, alors A est une propriété de sûreté pour le modèle en question (déduction).
- ▶ Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour induire des annotations pour le modèle (induction).



- ▶ $\text{VALS} = \text{LOCATIONS} \times \text{MEMORY}$
- ▶ $J(pc_0, v_0, pc, v) \stackrel{def}{=} \exists x_0, x \in \text{VALS}. I(x_0, x) \wedge x = (pc, v) \wedge x_0 = (pc_0, v_0)$ (deduction)
- ▶ $I(x_0, x) \stackrel{def}{=} \exists pc_0, pc \in \text{LOCATIONS}, v_0, v \in \text{MEMORY}. J(pc_0, v_0, pc, v) \wedge x = (pc, v) \wedge x_0 = (pc_0, v_0)$ (induction)



- ▶ $\text{VALS} = \text{LOCATIONS} \times \text{MEMORY}$
- ▶ $J(pc, v) \stackrel{\text{def}}{=} \exists x \in \text{VALS}. I(x) \wedge x = (pc, v)$ (deduction)
- ▶ $I(x) \stackrel{\text{def}}{=} \exists pc \in \text{LOCATIONS}, v \in \text{MEMORY}. J(pc, v) \wedge x = (pc, v)$ (induction)



- (1) $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow A(x) \ (I(x))$
- (2) $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{Next}^*(x_0, x) \Rightarrow R(x_0, x) \ (IR(x))$

Relations et définitions

$x = (\ell, v)$, $x_0 = (\ell_0, v_0)$, $I(x)$, $IR(x_0, x)$ et les annotations $P_\ell(v)$, $RP_\ell(v_0, v)$ sont liées ainsi :

- ▶ $I(x) \stackrel{\text{def}}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- ▶ $P_\ell(v) \stackrel{\text{def}}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge I(x))$
- ▶ $IR(x_0, x) \stackrel{\text{def}}{=} \exists \ell, v, v_0. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge RP_\ell(v_0, v))$
- ▶ $RP_\ell(v_0, v) \stackrel{\text{def}}{=} \exists x, x_0. (x, x_0 \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge IR(x_0, x))$

La transformation est fondée la relation de transition définie pour chaque couple d'étiquettes de contrôle qui se suivent est exprimée très simplement par la forme relationnelle suivante :

$$x \ r_{\ell, \ell'} \ x' \stackrel{\text{def}}{=} (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$$

- ▶ La transition de ℓ à ℓ' est possible, quand la condition $cond_{\ell,\ell'}(v)$ est vraie pour V et quand le contrôle est en ℓ ($pc = \ell$).
- ▶ Quand la transition est observée, les variables V sont transformées comme suit $v' = f_{\ell,\ell'}(v)$.
- ▶ La définition de la transition n'exprime aucune hypothèse liée à une stratégie d'exécution comme l'équité par exemple.
- ▶ $cond_{\ell,\ell'}(v) \wedge v' = f_{\ell,\ell'}(v)$ est une expression où les expressions $cond_{\ell,\ell'}(v)$ et $v' = f_{\ell,\ell'}(v)$ posent des questions de définition :
 - $DOM(\ell, \ell')(v) \stackrel{def}{=} DEF(cond_{\ell,\ell'}(v))(v) \wedge DEF(f_{\ell,\ell'}(v))$
 - $DEF(E(X))(x)$, signifie que l'expression $E(X)$ est définie pour x la valeur courante de X .
- ▶ Certaines transitions peuvent conduire à des catastrophes :
 - $DEF(X+1)(x) \stackrel{def}{=} x+1 \in D$ où D est le domaine de codage de X par exemple $D = -2^{31} \dots 2^{31}-1$ pour un codage sur 32 bits.
 - $DEF(T(I+1) < V)(t, x, v) \stackrel{def}{=} i+1 \in dom(t) \wedge v \in D \wedge t(i+1) \in D$

$$\begin{aligned}\ell &: P_\ell(v_0, v) \\ V &:= f_{\ell, \ell'}(V) \\ \ell' &: P_{\ell'}(v_0, v)\end{aligned}$$

Traduction

- ▶ $(pc = \ell \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$
- ▶ $cond_{\ell, \ell'}(v) \stackrel{def}{=} TRUE$

$$\begin{aligned}\ell_1 &: P_{\ell_1}(v_0, v) \\ \textbf{WHILE } B(V) \textbf{ DO} \\ &\ell_2 : P_{\ell_2}(v_0, v) \\ &\dots \\ &\ell_3 : P_{\ell_3}(v_0, v) \\ \textbf{END} \\ \ell_4 &: P_{\ell_4}(v_0, v)\end{aligned}$$

Traduction

$$\left\{ \begin{array}{l} pc = \ell_1 \wedge b(v) \wedge v' = v \wedge pc' = \ell_2 \\ pc = \ell_1 \wedge \neg b(v) \wedge v' = v \wedge pc' = \ell_4 \\ pc = \ell_3 \wedge b(v) \wedge v' = v \wedge pc' = \ell_2 \\ pc = \ell_3 \wedge \neg b(v) \wedge v' = v \wedge pc' = \ell_4 \end{array} \right.$$

Un programme P *remplit* un contrat ($pre, post$) :

- ▶ P transforme une variable v à partir d'une valeur initiale v_0 et produisant une valeur finale v_f : $v_0 \xrightarrow{P} v_f$
- ▶ v_0 satisfait pre : $pre(v_0)$ and v_f satisfait $post$: $post(v_0, v_f)$
- ▶ $pre(v_0) \wedge v_0 \xrightarrow{P} v_f \Rightarrow post(v_0, v_f)$

requires $pre(v_0)$

ensures $post(v_0, v_f)$

variables V

begin

$0 : P_0(v_0, v)$

instruction₀

...

$i : P_i(v_0, v)$

...

instruction _{$f-1$}

$f : P_f(v_0, v)$

end

- ▶ $pre(v_0) \wedge v = v_0 \Rightarrow P_0(v_0, v)$
- ▶ $pre(v_0) \wedge P_f(v_0, v) \Rightarrow post(v_0, v)$
- ▶ conditions sur les transitions ℓ, ℓ' à définir à partir des principes d'induction.

```
variables  $U, V$   
requires  $u_0, v_0 \in \mathbb{N}$   
ensures  $u_f + v_f = u_0 + v_0$   
begin  
  0 :  $u = u_0 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N}$   
   $U := U + 2$   
  1 :  $u = u_0 + 2 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N}$   
   $V := V - 2$   
  2 :  $u = u_0 + 2 \wedge v = v_0 - 2 \wedge u_0, v_0 \in \mathbb{N}$   
end
```

$$x = (pc, v) \text{ et } J(\ell_0, v_0, pc, v) \stackrel{\text{def}}{=} \left[\begin{array}{l} \wedge pc \in \text{LOCATIONS} \\ \wedge v \in \text{MEMORY} \\ \dots \\ \wedge pc = \ell \Rightarrow P_\ell(v_0, v) \\ \dots \end{array} \right.$$

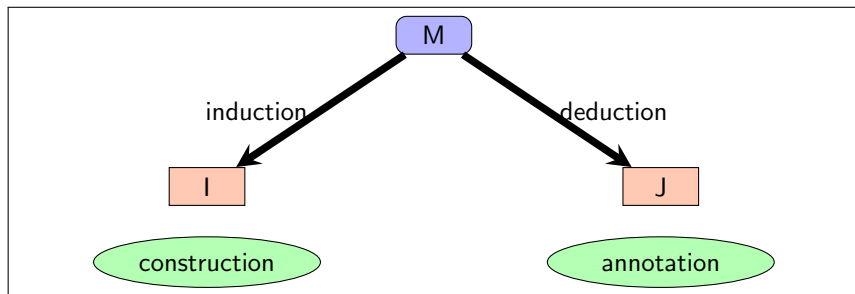
Soit $(Th(s, c), x, \text{VALS}, \text{Init}(x), \{r_0, \dots, r_n\})$ un modèle relationnel pour ce programme. Une propriété $A(x)$ est une propriété de sûreté pour P , si $\forall x_0, x \in \text{LOCATIONS} \times \text{MEMORY}. \text{Init}(x_0) \wedge x_0 \xrightarrow{\text{Next}} x \Rightarrow A(x)$. On sait que cette propriété implique qu'il existe une propriété d'état $I(x)$ telle que les trois propriétés sont vérifiées mais on applique cette vérification pour J :

$\forall x_0, x, x' \in \text{LOCATIONS} \times \text{MEMORY} :$

$$\left\{ \begin{array}{l} (1) \text{ Init}(x_0) \Rightarrow J(x_0, x_0) \\ (2) J(x_0, x) \Rightarrow A(x_0, x) \\ (3) \forall i \in \{0, \dots, n\} : J(x_0, x) \wedge x \text{ r}_i x' \Rightarrow J(x_0, x') \end{array} \right.$$

- $\forall i \in \{0, \dots, n\} : J(x) \wedge x \text{ r}_i x' \Rightarrow J(x')$ est équivalent à $J(x) \wedge (\exists i \in \{0, \dots, n\} : x \text{ r}_i x') \Rightarrow J(x')$

- ▶ Application de la correction du principe d'induction : si on vérifie les trois propriétés, alors A est une propriété de sûreté pour le modèle en question (déduction).
- ▶ Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour induire des annotations pour le modèle (induction).



- 1 $\text{Init}(0, u_0, v_0,) \Rightarrow J(0, u_0, v_0, 0, u_0, v_0) :$
 $pc = 0 \wedge u = u_0 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N} \Rightarrow J(0, u_0, v_0, 0, u_0, v_0) :$
- 2 $J(0, u_0, v_0, pc, u, v) \Rightarrow A(0, u_0, v_0, pc, u, v)$
 $J(pc, u, v) \Rightarrow (pc = 2 \Rightarrow u+v = u_0+v_0-2 \wedge u_0, v_0 \in \mathbb{N})$
- 3 $\forall i \in \{0, \dots, n\} : J(0, u_0, v_0, pc, u, v) \wedge x \ r_i \ pc', u', v' \Rightarrow$
 $J(0, u_0, v_0, pc', u', v')$
 $\left[\begin{array}{l} r01(pc, u, v, pc', u', v') \stackrel{def}{=} pc = 0 \wedge u' = u+2 \wedge pc' = 1 \wedge v' = v \\ r12(pc, u, v, pc', u', v') \stackrel{def}{=} pc = 1 \wedge v' = v-2 \wedge pc' = 2 \wedge u' = u \end{array} \right.$
 - $J(0, u_0, v_0, pc, u, v) \wedge r01(pc, u, v, pc', u', v') \Rightarrow J(pc', u', v')$
 - $J(0, u_0, v_0, pc, u, v) \wedge r12(pc, u, v, pc', u', v') \Rightarrow J(0, u_0, v_0, pc', u', v')$

lification 1 $[A \wedge (A \Rightarrow B)] \longrightarrow [A \wedge B]$

lification 2 $[A \wedge (B = C) \wedge D \Rightarrow E \wedge (B = F) \wedge G] \longrightarrow [A \wedge (B = C) \wedge D \Rightarrow E \wedge (C = F) \wedge G]$

lification 3 $[A \wedge (B = C) \wedge D \Rightarrow E \wedge (F = F) \wedge G] \longrightarrow [A \wedge (B = C) \wedge D \Rightarrow E \wedge TRUE \wedge G]$

lification 4 $[A \Rightarrow B \wedge TRUE \wedge C] \longrightarrow [A \Rightarrow B \wedge C]$



lification 5 $[A \wedge (B = C \Rightarrow U) \wedge (B = D \wedge B = C \Rightarrow V)] \wedge C \neq D \wedge E] \longrightarrow [A \wedge B = C \wedge U \wedge C \neq D \wedge E]$

$$J(pc, u, v) \wedge r01(pc, u, v, pc', u', v') \Rightarrow J(pc', u', v') \quad \mathbf{(1)}$$

$$\begin{pmatrix} \wedge pc \in \{0, 1, 2\} \\ \wedge u, v \in \mathbb{Z} \\ \wedge pc = 0 \Rightarrow u = u_0 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc = 1 \Rightarrow u = u_0 + 2 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc = 2 \Rightarrow u = u_0 + 2 \wedge v = v_0 - 2 \wedge u_0, v_0 \in \mathbb{N} \end{pmatrix} \wedge \begin{pmatrix} \wedge pc = 0 \\ \wedge u' = u + 2 \\ \wedge pc' = 1 \\ \wedge v' = v \end{pmatrix} \\ \Rightarrow \begin{pmatrix} \wedge pc' \in \{0, 1, 2\} \\ \wedge u', v' \in \mathbb{Z} \\ \wedge pc' = 0 \Rightarrow u' = u_0 \wedge v' = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc' = 1 \Rightarrow u' = u_0 + 2 \wedge v' = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc' = 2 \Rightarrow u' = u_0 + 2 \wedge v' = v_0 - 2 \wedge u_0, v_0 \in \mathbb{N} \end{pmatrix}$$

Utilisation de TLA Toolbox pour vérifier ces éléments : cours1.tla